✦ AI ASTROLOGER ✦

Vedic Knowledge Digitization & Neuro-Symbolic Reasoning Engine

---

MASTER PROJECT ROADMAP

Version 3.0 · 16 Phases · 52 Weeks

Hardware	Stack	Phases	Goal
MacBook Air M2(PC for Phase 13+)	Gemini Pro · Neo4jPython 3.11	16 Phases52 Weeks	Explainable AIJyotish Engine

✦ *The stars speak in logic. This system will learn to listen.* ✦

1. Core Philosophy

This is not a chatbot that looks up pre-written horoscopes. This is not a rule-matcher that searches IF Mars in 9th THEN fortune. This is an AI reasoning engine that thinks the way a great Jyotishi thinks — reasoning from first principles for any planetary combination, including ones never written in any classical text.

Old Approach	Your Approach
Astrology = Static Rules	Astrology = Symbolic Reasoning
IF Mars in 9th → fortune	Mars→action · 9th→dharma → fortune through effort
Fails on novel combinations	Reasons from first principles always
No explainability	Full traceable reasoning tree per prediction

---

2. Your Actual Setup

Hardware & Environment

Component	Details
Primary Machine	MacBook Air M2 — used for all development phases 0–12
Secondary (Later)	PC with NVIDIA GPU — needed only from Phase 13 (fine-tuning)
OS	macOS — no CUDA/Ubuntu needed until Phase 13
Code Editor	Codex (OpenAI AI code editor)
Python	3.11 via pyenv or Homebrew
Package Manager	pip + venv (no conda needed)

What You Already Have

Existing Assets — Do Not Rebuild
✓ Extraction prompt (v2.0) — multi-format JSON schema with rules/yogas/descriptions/calculations
✓ Chunking pipeline — PDF → chunks → Gemini Pro → structured JSON output
✓ Continuation protocol — handles large books across multiple API calls
✓ Stitching logic — merges all chunks into one unified output file
✓ 1-5 books already processed and validated

✓ 200-300 books downloaded locally and ready for processing

These are kept and wrapped cleanly into the new architecture. No rebuilding required — only reorganisation and adding the missing normalisation layer.

---

3. Clean 4-Module Architecture

All existing random scripts and notebooks get reorganised into this structure. Each module has one clear responsibility. Codex works best with this pattern — focused files with single responsibilities.

Project Folder Structure
ai-astrologer/
├── extractor/ # Your Gemini pipeline (cleaned up)
│ ├── chunker.py # PDF → chunks with metadata
│ ├── gemini_client.py # Gemini API wrapper
│ ├── prompt.py # Your extraction prompt v2.0
│ └── stitcher.py # Merges chunk outputs
├── normaliser/ # NEW — most critical missing piece
│ ├── ontology/ # JSON entity definitions
│ ├── normaliser.py # Surya → SUN, Lagna → HOUSE_1
│ └── validator.py # Validates extracted rules
├── storage/ # Database handlers
│ ├── neo4j_client.py # Knowledge graph operations
│ └── sqlite_client.py # Chunk index + metadata
├── pipeline/ # Orchestrates all 3 modules
│ └── run_book.py # Single command: book → Neo4j
├── reasoning/ # Phases 5-11 (built later)
├── data/ # All data (git-ignored)
│ ├── raw/ # Original PDFs — never modify
│ ├── extracted/ # JSON rule files per book
│ └── db/ # SQLite databases
├── AGENTS.md # Codex context file
├── .env # API keys — never commit

```
└── requirements.txt
```

The Clean Data Flow

Step	What Happens
1. PDF Input	Raw book PDF placed in data/raw/
2. Chunker	extractor/chunker.py splits into ~25 chunks with metadata
3. Gemini Extraction	extractor/gemini_client.py sends each chunk with your v2.0 prompt
4. Stitch	extractor/stitcher.py merges all chunk JSONs into one file
5. Normalise	normaliser/normaliser.py maps Surya→SUN, Lagna→HOUSE_1 etc
6. Validate	normaliser/validator.py flags unknown entities for review
7. Store	storage/neo4j_client.py loads rules as nodes into Neo4j graph
8. Index	storage/sqlite_client.py records book metadata + chunk index

---

4. AGENTS.md — Codex Context File

This file lives at the project root and tells Codex exactly what your project is and how it's structured. With this file, every Codex session starts with full context — no re-explaining architecture every time.

AGENTS.md Contents (create this file first)
AI Astrologer — Codex Context
What This Project Is
A Vedic astrology knowledge extraction and reasoning engine.
200-300 classical books → structured knowledge graph → explainable AI predictions.
Architecture: 4 Modules
- extractor/ : PDF chunking + Gemini Pro extraction (existing pipeline)
- normaliser/ : Maps Sanskrit/English synonyms to canonical ontology terms
- storage/ : Neo4j knowledge graph + SQLite chunk index
- pipeline/ : Orchestrates all modules for end-to-end book processing

Key Rules
- Clean code, minimal comments, type hints on all functions
- Single responsibility per file
- All entity names must be canonical (SUN not Surya) before Neo4j storage
- Never modify files in data/raw/
- All secrets in .env, never hardcoded

5. All 16 Phases at a Glance

Phase	Name	Key Deliverable
Phase 0	Foundation & Infrastructure	Dev environment on Mac + clean folder structure
Phase 1	Formal Ontology Design	Controlled vocabulary + synonym map + Neo4j schema
Phase 2+3	Extraction Pipeline (Combined)	All 200-300 books → normalised JSON rules in Neo4j
Phase 4	Knowledge Graph & Chart Ingestion	Swiss Ephemeris engine + chart subgraph in Neo4j
Phase 5	Core Logic Flow Engine	House→Lord→Placement chain reasoning
Phase 6	Planetary Strength Scoring	Numerical strength scores per planet
Phase 7	Yoga & Aspect Detection	Pattern detector for classical yoga library
Phase 8	House Influence Propagation	Mathematical influence flow across houses
Phase 9	Temporal Dynamics	Dasha + transit predictive engine
Phase 10	Divisional Chart Layer	D1/D9/D10/D12 multi-chart reasoning
Phase 11	Interpretation & NLG	Coherent narrative from logical fact set
Phase 12	Chart Dataset Validation	Statistical rule testing on real charts
Phase 13	Self-Learning Engine	ILP + pattern mining (switch to PC here)
Phase 14	Hybrid Neuro-Symbolic AI	Fine-tuned LLM constrained by knowledge graph
Phase 15	UI & Deployment	Streamlit dashboard + production-ready system

---

Phase 0 — Foundation & Infrastructure (Weeks 1–2)

Goal: Clean environment on Mac. Everything installed, tested, and working before any data work begins.

Mac Setup (No CUDA/Docker needed yet)

Installation Steps
1. Install Homebrew if not present: /bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
2. brew install python@3.11 git

3. Install Neo4j Desktop (native Mac app): neo4j.com/download — simplest option, no Docker
4. pip install neo4j chromadb google-generativeai pyswisseph fastapi uvicorn pandas
5. pip install pytest black ruff jupyter
6. pip install python-dotenv
7. Create project folder structure (see Section 3)
8. Create .env with GEMINI_API_KEY=your_key_here
9. Create AGENTS.md (see Section 4)
10. git init + create .gitignore (ignore .env, data/, __pycache__/)

Phase 0 Completion Checklist

Check	Verification
Python 3.11 installed	python3 --version shows 3.11.x
Neo4j Desktop running	Browser loads at localhost:7474
Gemini API working	Test script returns response
Folder structure created	All 4 module folders exist
AGENTS.md created	File exists at project root
.env configured	API key loads without error
Git initialised	git status shows clean repo
requirements.txt saved	pip freeze > requirements.txt done

Phase 1 — Formal Ontology Design (Weeks 3–4)

Goal: Complete controlled vocabulary before any book is processed. This is the DNA of the entire system. Every extracted rule, every Neo4j node, every reasoning step depends on getting this right first.

Entity Types to Define

Entity	What to Define
Planets (9 Grahas)	Canonical name, all Sanskrit/English synonyms, nature, exaltation, debilitation, friendships, dasha years
Signs (12 Rashis)	Canonical name, synonyms, ruler, element, modality, exaltation/debilitation planets

Houses (12 Bhavas)	Canonical name, synonyms (Lagna, Ascendant, Tanu Bhava etc), primary/secondary meanings, karaka
Nakshatras (27)	Canonical name, synonyms, lord, deity, shakti, pada breakdown
Yogas (50+)	Name, formation conditions as machine-readable logic, classical effects, source text
Dignity Types	Exaltation, debilitation, moolatrikona, own sign, friend/enemy — with strength bonus/penalty values
Dashas	Vimshottari periods and sub-periods with lords
Abstract Concepts	Wealth, marriage, career, health, children etc — mapped to house/planet karakas

The Normaliser — Most Critical Function

Every entity name from every extracted rule passes through normalise() before being stored. This is what prevents 'Surya', 'Ravi', and 'Sun' from becoming three separate things in your knowledge graph.

normaliser/normaliser.py — Key Function
normalise('Surya') → 'SUN'
normalise('Ravi') → 'SUN'
normalise('Lagna') → 'HOUSE_1'
normalise('Ascendant') → 'HOUSE_1'
normalise('Guru') → 'JUPITER'
normalise('Brihaspati') → 'JUPITER'
normalise('Kuja') → 'MARS'
normalise('Mesha') → 'ARIES'
normalise('unknown') → None ← flagged for human review

Synonym Coverage Target

Planet	All Synonyms to Map
SUN	Sun, Surya, Ravi, Arka, Aditya, Bhanu, Dinakara, Bhaskar
MOON	Moon, Chandra, Soma, Indu, Nisha, Mriganka, Himanshu, Shashi
MARS	Mars, Mangala, Kuja, Bhouma, Angaraka, Lohitanga, Ara
MERCURY	Mercury, Budha, Saumya, Gna, Kumar
JUPITER	Jupiter, Guru, Brihaspati, Jeeva, Devaguru, Angiras

VENUS	Venus, Shukra, Bhrigu, Kavi, Sita, Usanas
SATURN	Saturn, Shani, Sanaischara, Manda, Krura, Asita, Arkaja
RAHU	Rahu, Sarpa, Dragon's Head, North Node, Tamas, Svarbhanu
KETU	Ketu, Dragon's Tail, South Node, Sikhi, Mokshakaraka

Phase 1 Checklist

Check	Verification
All 9 planets defined in planets.json	9 planets load without errors
All 12 signs defined in signs.json	12 signs load without errors
All 12 houses defined in houses.json	12 houses load without errors
All 27 nakshatras defined	27 nakshatras load without errors
50+ yogas with machine-readable conditions	Yoga JSON schema validates
500+ synonyms mapped	normalise() passes all test cases
All entities loaded into Neo4j	MATCH (n) RETURN count(n) → 60+ nodes
Validator working	validate_rule() passes on sample rules

Phase 2+3 — Extraction Pipeline (Weeks 5–16)

These two phases are merged because your existing pipeline already handles both chunking and extraction together. The work here is: wrap your existing code cleanly, add the normalisation step, and process all 200-300 books.

What Changes vs Your Current Pipeline

Current	New (Clean Version)
Random scripts in files	4 clean modules with single responsibilities
Raw string entity names (Surya, Ravi)	All normalised to canonical (SUN) before storage
Output: flat stitched JSON file	Output: flat file + loaded into Neo4j + indexed in SQLite
No confidence tracking	Same rule from 2+ books → confidence score increases
No unknown entity flagging	Entities not in ontology flagged for review automatically
Manual book-by-book runs	pipeline/run_book.py handles one command per book

Processing Priority

Tier	Books	Why First
Tier 1	Saravali, Brihat Jataka, Phala Deepika, Brihat Parashara Hora Shastra	Foundational texts — most rules cited across other books
Tier 2	Major Nadi Granthas, Jataka Parijata, Sarvartha Chintamani	High rule density, well-known sources
Tier 3	Commentaries, regional texts, personal research notes	Supplementary — process after Tier 1+2 complete

Extraction Schema — Keep Your v2.0 Prompt

Your existing extraction prompt is well-designed and produces the right output structure. The only addition needed is a post-processing normalisation step. The schemas (Format A through H) remain unchanged.

Key Schema Reminder — What Each Format Captures
Format A (rules) — IF [condition] THEN [result] with impact/intensity/probability
Format B (descriptions) — Inherent qualities of planets/signs/houses
Format C (calculations) — Step-by-step procedures and formulas

Format H (yogas)	— Named combinations with machine-readable formation logic
Formats D/E/F/G	— Context, cultural notes, case studies, references

Cross-Book Confidence Scoring

When the same rule appears in multiple books, its confidence score increases. This is tracked automatically during Neo4j ingestion:

Occurrences	Confidence Level
Found in 1 book	Low — flagged as single-source
Found in 2 books	Medium — reasonable confidence
Found in 3+ books	High — classical consensus
Found in 5+ books	Very High — foundational rule

Phase 4 — Knowledge Graph & Chart Ingestion (Weeks 17–20)

Build the chart calculation engine and complete Neo4j graph structure so any birth chart can be processed into a queryable subgraph.

Neo4j Node Types

Node Label	What It Represents
Planet	9 grahas with all attributes (from Phase 1 ontology)
Sign	12 rashis with element/modality/ruler
House	12 bhavas with meanings and karaka
Nakshatra	27 nakshatras with lord/deity/shakti
Rule	Each extracted rule — linked to triggering planets/houses
Yoga	Named combinations — linked to formation conditions
Chart	A specific birth chart — contains planet placements
Concept	Abstract themes: wealth, marriage, career, health etc

Chart Calculation Engine

pyswisseph (Swiss Ephemeris Python binding) calculates accurate planet positions for any birth date/time/place. Install: pip install pyswisseph

What the Chart Engine Computes
→ Planet positions (sign + degree + nakshatra pada)
→ House cusps (Whole Sign primary for Jyotish)
→ House lords (which planet rules which house)
→ Dignities (exalted / debilitated / own sign / friend / enemy)
→ Aspects (sign-based Jyotish drishti + special aspects for Mars/Jupiter/Saturn)
→ Retrograde status
→ Combustion (within 6 degrees of Sun)

Phases 5–8 — The Reasoning Engine (Weeks 21–30)

This is the heart of the project. These four phases together form the symbolic reasoning engine that makes the system think like a Jyotishi rather than look up pre-written rules.

Phase 5 — Core Logic Flow Engine (Weeks 21–24)

Graph traversal reasoning through causal chains. Not rule lookup — logical propagation.

Step	Logic
1. House Meaning	10th house = career / public life / authority
2. Find House Lord	10th lord = Saturn (Aquarius on 10th cusp)
3. Lord's Placement	Saturn in 3rd = communication, skills, effort
4. Lord's Nature	Saturn = discipline, structure, persistence
5. Aspects on Lord	Jupiter aspects Saturn = wisdom, advisory added
6. Strength Check	Saturn score 6.8 = moderately strong
7. Synthesis	Career through disciplined communication / analytical work

Key insight: Mars in 5th aspected by Mercury — no such rule written in any book. Mars (action) + 5th (intelligence/creativity) + Mercury aspect (intellect/analysis) = strategic intelligence. Pure logic, no book needed.

Phase 6 — Planetary Strength Scoring (Weeks 25–26)

Factor	Score Modifier
Exaltation	+5
Own Sign (Swakshetra)	+4
Moolatrikona	+3.5
Friendly Sign	+2
Kendra Placement (1,4,7,10)	+2
Trikona Placement (1,5,9)	+1.5
Benefic Aspect (Jupiter/Venus)	+1
Enemy Sign	-2
Trik House (6,8,12)	-1
Malefic Aspect (Saturn/Mars)	-1
Combustion (within 6° of Sun)	-3
Debilitation	-5

Phase 7 — Yoga & Aspect Detection (Weeks 27–28)

Yoga conditions stored as Cypher query templates in Neo4j. Pattern matcher runs against chart subgraph. Each matched yoga references its classical source and confidence level.

Phase 8 — House Influence Propagation (Weeks 29–30)

The most advanced reasoning layer. Instead of treating houses independently, models how influence radiates across the chart via lord placements and aspects — using a PageRank-style propagation algorithm. Gives 10x richer interpretations.

Phases 9–11 — Temporal, Divisional & Narrative (Weeks 31–37)

Phase 9 — Temporal Dynamics: Dashas & Transits (Weeks 31–33)

Layer	What It Does

Natal Promise	Mars in 9th → potential for fortune through effort
Dasha Activation	Mars dasha period → natal promise becomes active
Transit Trigger	Jupiter transiting 9th → benefic amplification
Synthesis	Time-specific prediction window identified

Phase 10 — Divisional Chart Layer (Weeks 34–35)

Chart	Domain
D1 — Rashi	General life — primary chart (built in Phase 4)
D9 — Navamsa	Dharma, marriage, inner planetary strength
D10 — Dashamsha	Career, profession, public reputation
D12 — Dwadashamsha	Parents, ancestral karma
D7 — Saptamsha	Children, progeny
D60 — Shashtyamsha	Karma — most subtle divisional

Phase 11 — Interpretation Synthesis & NLG (Weeks 36–37)

Converts the logical fact set into coherent human-readable interpretation. Phase 11 uses template-based NLG. Phase 14 upgrades this with a fine-tuned LLM — but the LLM output is always grounded in the symbolic fact set to prevent hallucination.

Example Reasoning Tree Output
PREDICTION: Strong career in structured communication or analytical fields
——
Step 1 → 10th house = career / public life
Step 2 → 10th lord = Saturn (Aquarius on 10th cusp)
Step 3 → Saturn in 3rd house = communication, effort, skills
Step 4 → Saturn nature = discipline, structure, persistence
Step 5 → Jupiter aspects Saturn = wisdom and advisory capacity added
Step 6 → Saturn strength score: 6.8 / 10 (own sign, kendra from Moon)
Step 7 → D10 check: Saturn strong in Dashamsha — career reinforced
——
Sources: Saravali Ch.12 · Brihat Jataka Ch.8

Phases 12–15 — Validation, Learning & Deployment (Weeks 38–52)

Phase 12 — Chart Dataset Validation (Weeks 38–39)

Turn astrology into data-driven research. Statistically test classical rules against real chart datasets. This is what makes the project academically significant.

Step	Detail
Collect datasets	500-1000+ birth records per category: entrepreneurs, artists, academics, politicians
Test a rule	Sun strong in 10th → public recognition — measure frequency vs control group
Output	Observed frequency, control frequency, p-value, confidence level
Result	High-confidence rules upgraded in knowledge graph automatically

Phase 13 — Self-Learning Engine (Weeks 40–43)

Switch to PC at Phase 13
This phase involves heavy computation: frequent subgraph mining over 50,000+ rules,
Inductive Logic Programming (ILP) over large chart datasets, and continuous validation loops.
MacBook Air M2 will be insufficient. Set up Ubuntu 22.04 + NVIDIA GPU on PC before starting.

The system discovers new rules and refines existing ones. Patterns found in 3+ independent texts become candidate yogas — reviewed by human expert before promotion. ILP learns rules from chart examples using your ontology as background knowledge.

Phase 14 — Hybrid Neuro-Symbolic AI (Weeks 44–47)

Fine-tune Phi-3 Mini (3.8B) with LoRA on your chart→interpretation pairs. The LLM generates language. The symbolic engine provides logic. Neither works as well without the other. LLM output is always constrained by the symbolic fact set — any entity mentioned must exist in the chart's fact set.

Phase 15 — UI & Deployment (Weeks 48–52)

Component	Technology

UI	Streamlit — fast to build, sufficient for 1-3 users
API	FastAPI — async Python, serves predictions and reasoning trees
Graph Viz	Pyvis for knowledge graph display
Chart Rendering	Custom SVG birth chart with house grid
Deployment	Local — all services running on your machine

---

6. Complete Technology Stack

Mac Phase (0–12)

Tool	Purpose
Python 3.11	Core language
Gemini Pro API	Primary extraction LLM — best quality for structured JSON output
Neo4j Desktop	Knowledge graph — native Mac app, no Docker needed
ChromaDB	Vector store for semantic search (pip install)
pyswisseph	Accurate Vedic chart calculations
FastAPI + Uvicorn	API layer
Streamlit	UI for personal use
SQLite	Chunk index + book metadata
Codex	AI-assisted development throughout
Git	Version control (no DVC needed for Mac phase)

PC Phase (13–15)

Tool	Purpose
Ubuntu 22.04 LTS	OS for GPU workloads
NVIDIA CUDA 12.x	GPU acceleration
Ollama + Mixtral 8x7B	Local LLM inference
PyTorch 2.x + CUDA	Deep learning backbone
HuggingFace Transformers + PEFT	LoRA fine-tuning for Phi-3 Mini

Aleph/Popper ILP	Inductive Logic Programming for rule discovery
Docker Compose	Service orchestration on PC

---

7. Success Criteria

Technical

- System reasons correctly from first principles for any planetary combination — including ones never written in any book
- Full traceable reasoning tree attached to every single prediction
- Temporal engine (dasha + transit) generates coherent time-based predictions
- API responds in under 5 seconds per chart on local hardware
- Full pipeline (book PDF → prediction) is reproducible end-to-end

Knowledge

- All 200-300 texts digitised, extracted, normalised, and stored in Neo4j
- 50,000+ atomic rules with cross-text confidence scores
- Chart dataset validation engine has statistically tested 100+ classical rules
- Divisional chart layer (D9/D10/D12) adds depth matching professional astrologers

Research

- Self-learning engine has discovered and validated at least 10 novel rule combinations
- House Influence Propagation Model demonstrates quantifiable inter-house energy flow
- Knowledge graph is a publishable resource for Vedic astrology AI research
- Statistical validation database is the first data-driven Jyotish research corpus

---

✦ ***Infrastructure ready. Ontology complete. Knowledge extraction begins.*** ✦

AI Astrologer · Master Roadmap v3.0 · 16 Phases · 52 Weeks